

# Programmazione Web e Mobile con Laboratorio

Lezione 13 - Node.js 2  
(Express.js e Middleware)

# Express.js

- È un framework web per Node.js
- Progettato per creare applicazioni web e API
- Consente di aggiungere funzionalità (logging, autenticazione, ...) tramite middleware
- Supporta un sistema di routing per gestire le richieste HTTP in base a vari percorsi e metodi (GET, POST, PUT, DELETE, ....)
- Integrabile con vari motori di template (Pug, EJS, ...) per generare pagine HTML dinamiche

# Application Programming Interface (API)

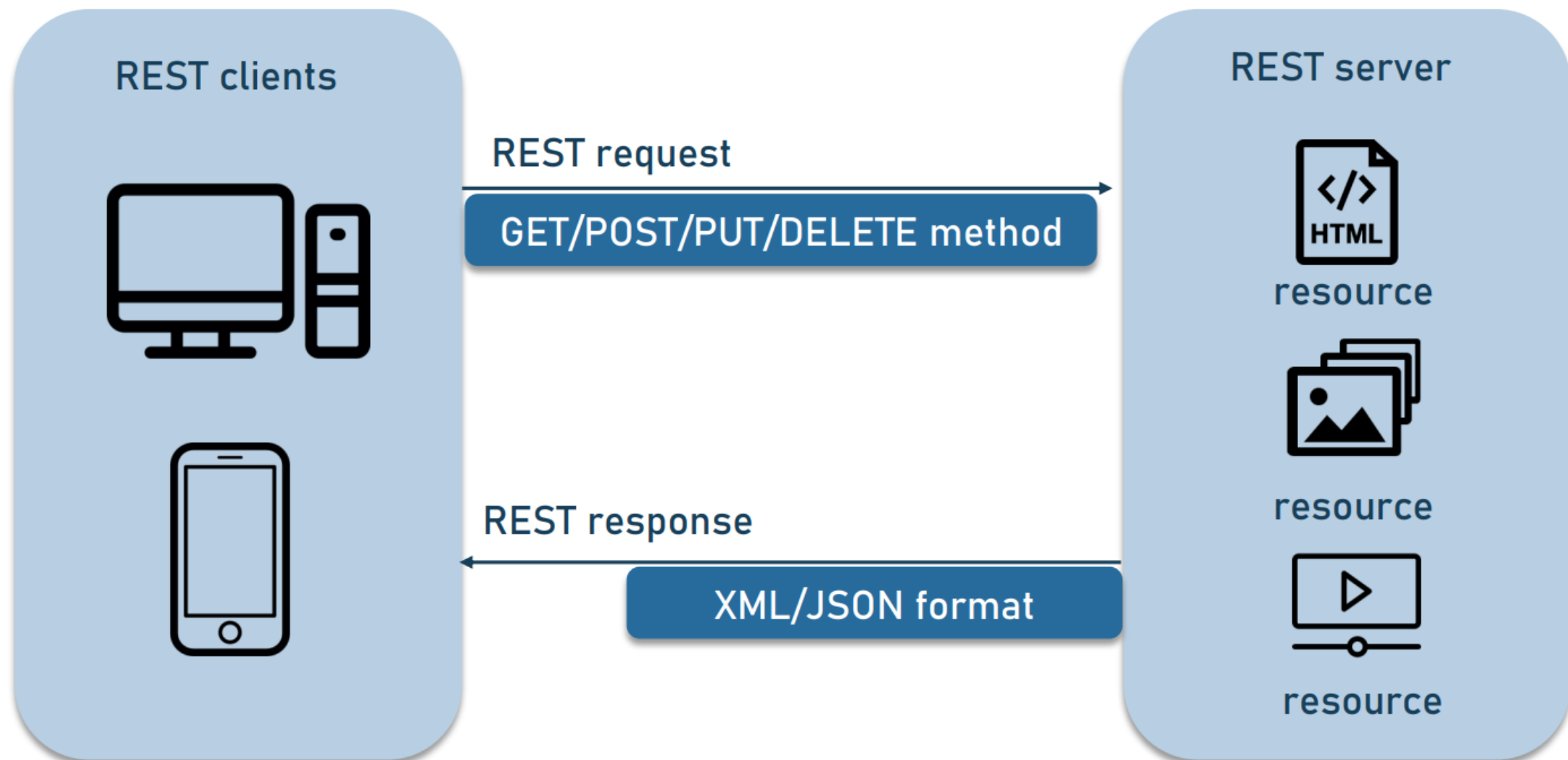
- Insieme protocolli che permettono a diverse applicazioni di comunicare tra loro
- Tipi di API: di Sistema, di Libreria, Web
- API Web: permettono la comunicazione tra server e client, ad es. tramite HTTP
- Componenti di un'API Web:
  - Endpoint (URL ai quali le richieste possono essere inviate)
  - Metodi HTTP (GET, POST, PUT, DELETE)
  - Headers (informazioni aggiuntive inviate con una richiesta o una risposta HTTP)
  - Body (contenuto della richiesta o risposta, solitamente in formato JSON o XML)

# REpresentational State Transfer (REST)

- Stateless: ogni richiesta deve contenere tutte le informazioni necessarie, senza che il server mantenga lo stato tra le richieste
- Client-Server: il client gestisce l'interfaccia utente, mentre il server si occupa dell'archiviazione e della logica di business
- Cacheable: le risposte possono essere memorizzate nella cache per ridurre le chiamate al server e migliorare le prestazioni
- Uniform Interface: le API RESTful usano metodi HTTP standard (GET, POST, PUT, DELETE) per operare sulle risorse in modo uniforme
- Layered System: l'architettura può avere più livelli (proxy, bilanciatori di carico) senza che il client ne sia consapevole

# REpresentational State Transfer (REST)

## REST API IN ACTION



# Metodi HTTP principali

- **GET:**
  - Utilizzato per leggere o recuperare dati, es. ottenere un elenco di prodotti
  - I dati vengono passati in chiaro nell'URL
  - È idempotente (chiamate ripetute restituiscono lo stesso risultato)
- **POST:**
  - Utilizzato per inviare dati e creare nuove risorse, es. creare un nuovo prodotto
  - I dati vengono inviati nel corpo della richiesta HTTP (non visibili direttamente)
  - Non è idempotente (chiamate ripetute possono causare effetti collaterali)

# Esempio CON Express.js

```
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send('Benvenuto nella home page');
});

app.get('/time', (req, res) => {
  res.send(`Ora corrente: ${new Date().toLocaleTimeString()}`);
});

app.get('/contact', (req, res) => {
  res.send('Contatto: email@example.com');
});

app.use((req, res) => {
  res.status(404).send('404 - Risorsa non trovata');
});

app.listen(port, () => {
  console.log(`Server avviato su http://127.0.0.1:${port}/`);
});
```

# Esempio SENZA Express.js

```
const http = require('http');
const hostname = '127.0.0.1';
const port = 3000;

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/plain');

  if (req.url === '/') {
    console.log(req.method);
    res.end('Benvenuto nella home page');
  }
  else if (req.url === '/time') {
    res.end(`Ora corrente: ${new Date().toLocaleTimeString()}`);
  }
  else if (req.url === '/contact') {
    res.end('Contatto: email@example.com');
  }
  else {
    res.statusCode = 404;
    res.end('404 - Risorsa non trovata');
  }
});

server.listen(port, hostname, () => {
  console.log(`Server running at http://${hostname}:${port}/`);
});
```

- Codice meno leggibile
- Gestione delle rotte meno chiara
- No middleware per errore 404
- No gestione automatica per header e status code

# Esercizio 1

- Passo 1: creare una nuova directory e inicializzarla

```
npm init -y
```

- Passo 2: installare Express.js

```
npm install express
```

- Passo 3: creare il file principale del server app.js

```
const express = require('express');
const app = express();
const port = 3000;

app.get('/', (req, res) => {
  res.send('Benvenuto nella home page');
});

app.get('/time', (req, res) => {
  res.send(`Ora corrente: ${new Date().toLocaleTimeString()}`);
});

app.get('/contact', (req, res) => {
  res.send('Contatto: email@example.com');
});

app.listen(port, () => {
  console.log(`Server avviato su http://127.0.0.1:${port}/`);
});
```

# res in Express.js

- `res.status(code)`: imposta il codice di stato della risposta HTTP
- `res.send(body)`: invia una risposta HTTP con il corpo specificato
- `res.json(obj)`: invia una risposta JSON
- `res.sendFile(path, [options], [callback])`: invia il file specificato al client
- `res.on(event, callback)`: ascolta eventi sulla risposta HTTP
- `res.cookie(name, value, [options])`: imposta un cookie
- `res.clearCookie(name, [options])`: rimuove un cookie
- ...

# Esercizio 1 (cont.)

- Passo 4: aggiungere una route che prende un parametro **name** da una query e risponde con un saluto personalizzato

```
app.get('/greet', (req, res) => {  
  const name = req.query.name || 'Ospite';  
  res.send(`Ciao, ${name}! Benvenuto nel nostro sito.`);  
});
```

- Passo 5: aggiungere una route che utilizza i parametri URL che prende il parametro **name** direttamente dall'URL e risponde con un saluto personalizzato

```
app.get('/greet2/:name', (req, res) => {  
  const name = req.params.name;  
  res.send(`Ciao, ${name}! Benvenuto nel nostro sito.`);  
});
```

# req.params vs req.query

La differenza principale tra req.params e req.query in Express.js riguarda il modo in cui i dati vengono passati nell'URL

| Caratteristica  | req.params                       | req.query                             |
|-----------------|----------------------------------|---------------------------------------|
| Dove si trova?  | Parte del percorso (/greet/name) | Dopo il ? nell'URL (/greet?user=name) |
| Obbligatorio?   | Sì, fa parte della route         | No, è opzionale                       |
| Tipico utilizzo | ID di utenti, prodotti, post     | Filtri, paginazione, ricerca          |

# req in Express.js

- req.params: parametri dell'URL (es. /users/:id → req.params.id)
- req.query: parametri della query (es. ?name=John → req.query.name)
- req.body: contiene il corpo della richiesta HTTP
- req.headers: contiene gli header della richiesta HTTP
- req.cookies: contiene i cookie inviati dal client
- req.method: restituisce il metodo HTTP della richiesta
- req.ip: restituisce l'indirizzo IP del client
- ...

# Esercizio 2

- Creare un server web utilizzando Express.js che serva pagine HTML diverse in base alla richiesta dell'utente
- Rotte da implementare:
  - `/`: index.html
  - **`/about`**: about.html
  - **`/contact`**: contact.html
- (Soluzione nella prossima slide)

```
const express = require('express');

const app = express();
const PORT = 3000;

app.get('/', (req, res) => res.sendFile(__dirname + '/index.html'));
app.get('/about', (req, res) => res.sendFile(__dirname + '/about.html'));
app.get('/contact', (req, res) => res.sendFile(__dirname + '/contact.html'));

app.listen(PORT, () => {
  console.log(`Server in ascolto su http://localhost:${PORT}`);
});
```

**\_\_dirname** è una variabile globale in Node.js che rappresenta il percorso assoluto della cartella in cui si trova il file attualmente in esecuzione

# Middleware in Express.js

- È una funzione che intercetta e gestisce le richieste HTTP prima che raggiungano la loro destinazione finale
- Può eseguire operazioni come l'autenticazione, la gestione degli errori, il logging, ecc.
- Un middleware ha accesso a:
  - **req**: l'oggetto richiesta
  - **res**: l'oggetto risposta
  - **next**: funzione che passa il controllo al middleware successivo

# Tipi di Middleware

- Built-in Middleware (già inclusi in Express.js)
  - `express.static()`: per servire file statici
  - `express.json()` : per elaborare richieste con corpo JSON
  - `express.urlencoded()`: per elaborare dati nei form
- Custom Middleware (scritti manualmente)
- Third-Party Middleware (moduli esterni)

# Esercizio 2 (cont.)

- Estendere l'esempio precedente aggiungendo un middleware custom per gestire gli errori 404

```
const express = require('express');

const app = express();
const PORT = 3000;

app.get('/', (req, res) => res.sendFile(__dirname + '/index.html'));
app.get('/about', (req, res) => res.sendFile(__dirname + '/about.html'));
app.get('/contact', (req, res) => res.sendFile(__dirname + '/contact.html'));

// Middleware per gestire errori 404 (pagina non trovata)
app.use((req, res) => {
  res.status(404).sendFile(__dirname + '/404.html');
});

app.listen(PORT, () => {
  console.log(`Server in ascolto su http://localhost:${PORT}`);
});
```

# Esercizio 2 (cont.)

- Aggiungere un middleware che, ad ogni richiesta ricevuta dal server, stampa il metodo HTTP, l'URL della richiesta e la durata dell'operazione

```
// Middleware per loggare il metodo HTTP, l'URL e la durata della richiesta
app.use((req, res, next) => {
  const start = Date.now(); // Tempo di inizio
  res.on('finish', () => { // Eseguito quando la risposta viene inviata
    const duration = Date.now() - start;
    console.log(`${req.method} ${req.url} - ${duration}ms`);
  });
  next();
});
```

- Questo middleware deve essere dichiarato prima delle altre route, altrimenti non verrebbe eseguito per le richieste già gestite
- Altri middleware (ad esempio quello per gli errori 404) devono essere dichiarati dopo le route per gestire ciò che non è stato intercettato prima

# Esercizio 3

- Invece di gestire ogni rotta manualmente con `res.sendFile()`, usare `express.static()` per gestire i file statici
- Express.js cercherà i file direttamente nella cartella `public/` senza bisogno di dichiarare ogni route

```
const express = require('express');

const app = express();
const PORT = 3000;

app.use(express.static(__dirname + '/public'));

app.use((req, res) => {
  res.status(404).sendFile(__dirname + '/public/404.html');
});

app.listen(PORT, () => {
  console.log(`Server in ascolto su http://localhost:${PORT}`);
});
```

# HTML statico e dinamico

- **/mia\_pagina può restituire HTML dinamico**
- Vantaggi:
  - URL più puliti e leggibili
  - Permette di modificare il formato della risposta senza cambiare l'URL (HTML, JSON, ...)
- Svantaggi:
  - Il server deve sapere che /mia\_pagina restituisce una pagina HTML
  - Può generare confusione con le API (/mia\_pagina potrebbe essere un endpoint JSON invece di un file HTML)

- **/mia\_pagina.html restituisce un file statico**
- Vantaggi:
  - È chiaro che si tratta di un file HTML statico
  - Il server lo restituisce direttamente senza elaborazione
- Svantaggi:
  - Se cambia il formato (es. da HTML a JSON), bisogna cambiare l'URL
  - Meno flessibile per applicazioni dinamiche

# HTML statico e dinamico

- `/mia_pagina` può restituire HTML dinamico
- Vantaggi:
  - URL più puliti e leggibili
  - Permette di modificare il formato della risposta senza cambiare l'URL (HTML, JSON, ...)
- Svantaggi:
  - Il server deve sapere che `/mia_pagina` restituisce una pagina HTML
  - Può generare confusione con le API (`/mia_pagina` potrebbe essere un endpoint JSON invece di un file HTML)

- 
- `/mia_pagina.html` restituisce un file statico
  - Vantaggi:
    - È chiaro che si tratta di un file HTML statico
    - Il server lo restituisce direttamente senza elaborazione
  - Svantaggi:
    - Se cambia il formato (es. da HTML a JSON), bisogna cambiare l'URL
    - Meno flessibile per applicazioni dinamiche

# Cookie-parser

- È un middleware per Express.js che permette di leggere e gestire i cookie nelle richieste HTTP
- Il cookie viene impostato con il metodo `res.cookie(name, value, options)`
- Cookie-parser popola l'oggetto `req.cookies` per consentirne l'utilizzo (solo `req.cookies` richiede `cookie-parser`)
- È un modulo npm e va installato

```
npm install cookie-parser
```

# Esercizio 4

- Creare un server Express.js che gestisca i cookie utilizzando il middleware cookie-parser. Il server dovrà avere tre route:
  - **/set-cookie**: imposta un cookie
  - **/get-cookie**: legge il cookie
  - **/clear-cookie**: cancella il cookie
- Il cookie deve avere un tempo di scadenza (maxAge) e deve essere accessibile solo tramite HTTP (httpOnly)

```
const express = require('express');
const cookieParser = require('cookie-parser');

const app = express();
const PORT = 3000;

app.use(cookieParser());

// Imposta un cookie
app.get('/set-cookie', (req, res) => {
  res.cookie('username', 'Alice', { maxAge: 900000, httpOnly: true });
  res.send('Cookie impostato!');
});

// Legge i cookie
app.get('/get-cookie', (req, res) => {
  res.send(`Cookie ricevuti: ${JSON.stringify(req.cookies)}`);
});

// Cancella un cookie
app.get('/clear-cookie', (req, res) => {
  res.clearCookie('username');
  res.send('Cookie cancellato!');
});

app.listen(PORT, () => {
  console.log(`Server in ascolto su http://localhost:${PORT}`);
});
```

- L'opzione **maxAge** definisce per quanto tempo un cookie sarà valido in millisecondi prima di essere automaticamente eliminato dal browser
  - In alternativa, è possibile usare l'opzione **expires**, che imposta invece una data di scadenza precisa
- L'opzione **httpOnly: true** rende il cookie accessibile solo tramite il server e non modificabile da JavaScript lato client
  - Previene attacchi XSS (Cross-Site Scripting), dato che gli script malevoli non possono leggere il cookie
- L'operazione **JSON.stringify(req.cookies)** serve per convertire i cookie ricevuti in una stringa JSON più leggibile

# Esercizio 5

- Creare un server Express.js che distribuisce due categorie di pagine: pubbliche e private
- Le pagine private sono distribuite solamente previa verifica di un cookie
- Routes da prevedere:
  - /login - imposta un cookie
  - /logout - elimina il cookie
  - /restricted - controlla il cookie e restituisce la pagina privata